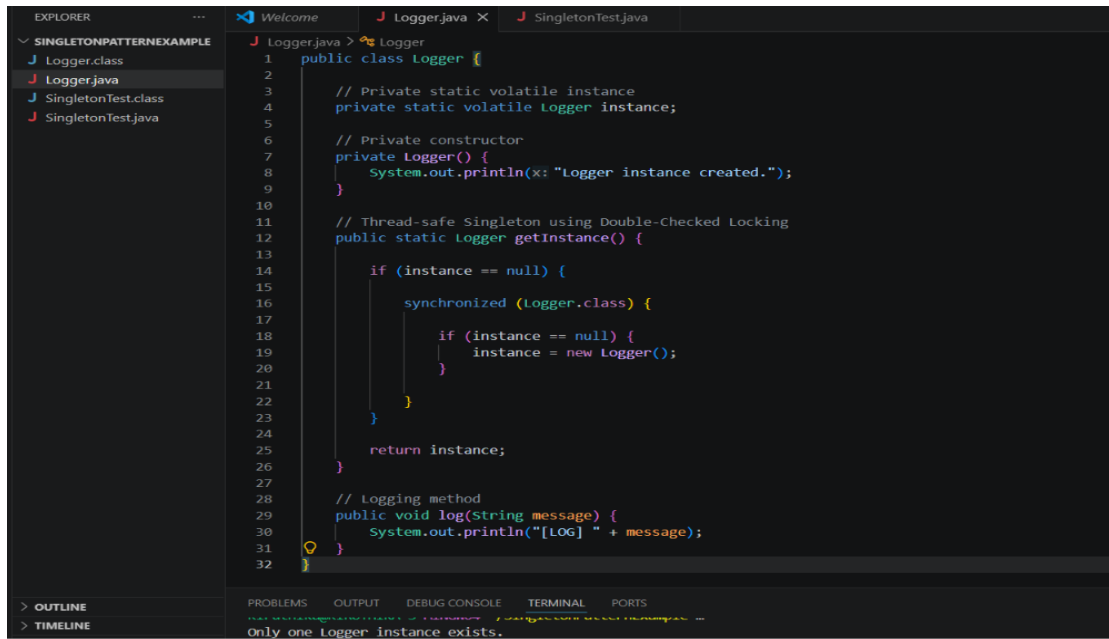


DIGITAL NURTURE 5.0 DEEPSKILLING

Week 1 – HANDS-ON

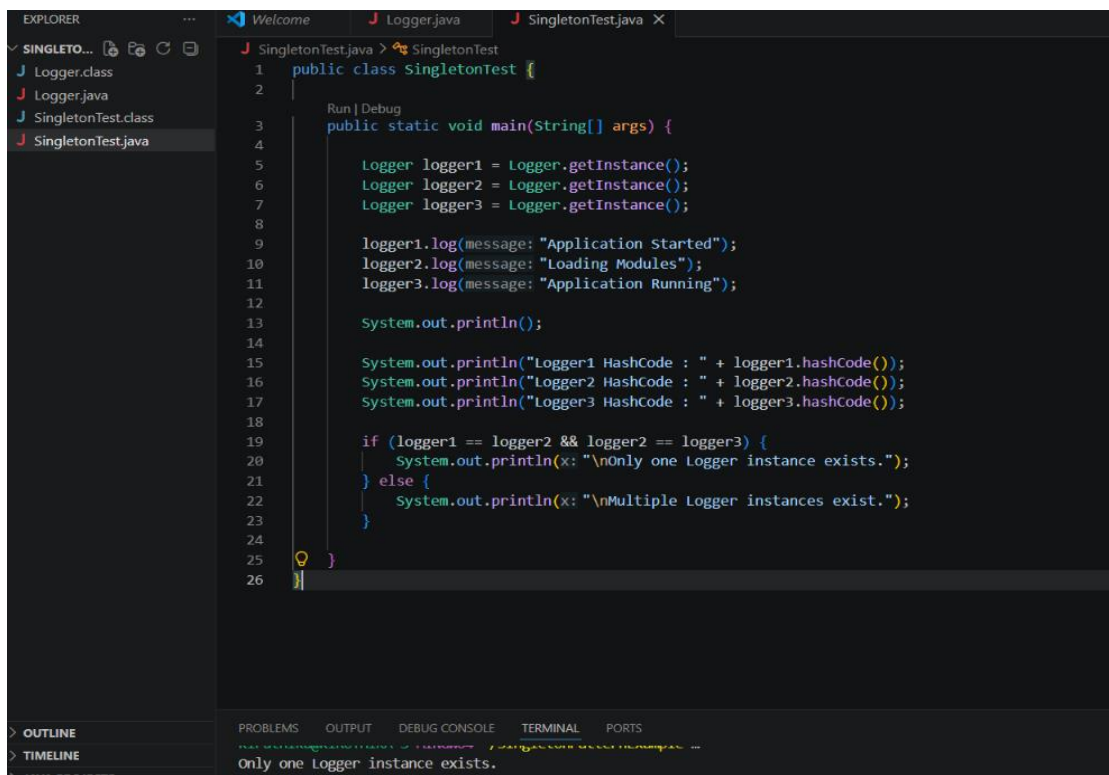
Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern



```
1 public class Logger {
2
3     // Private static volatile instance
4     private static volatile Logger instance;
5
6     // Private constructor
7     private Logger() {
8         System.out.println(x: "Logger instance created.");
9     }
10
11     // Thread-safe Singleton using Double-Checked Locking
12     public static Logger getInstance() {
13
14         if (instance == null) {
15             synchronized (Logger.class) {
16                 if (instance == null) {
17                     instance = new Logger();
18                 }
19             }
20         }
21
22         return instance;
23     }
24
25     // Logging method
26     public void log(String message) {
27         System.out.println("[LOG] " + message);
28     }
29 }
```

Only one Logger instance exists.



```
1 public class SingletonTest {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Logger logger1 = Logger.getInstance();
7         Logger logger2 = Logger.getInstance();
8         Logger logger3 = Logger.getInstance();
9
10        logger1.log(message: "Application Started");
11        logger2.log(message: "Loading Modules");
12        logger3.log(message: "Application Running");
13
14        System.out.println();
15
16        System.out.println("Logger1 hashCode : " + logger1.hashCode());
17        System.out.println("Logger2 hashCode : " + logger2.hashCode());
18        System.out.println("Logger3 hashCode : " + logger3.hashCode());
19
20        if (logger1 == logger2 && logger2 == logger3) {
21            System.out.println(x: "\nOnly one Logger instance exists.");
22        } else {
23            System.out.println(x: "\nMultiple Logger instances exist.");
24        }
25    }
26 }
```

Only one Logger instance exists.

```
1 public class SingletonTest {
2
3     Run | Debug
    public static void main(String[] args) {
4
5         Logger logger1 = Logger.getInstance();
6         Logger logger2 = Logger.getInstance();
7         Logger logger3 = Logger.getInstance();
8
9         logger1.log(message: "Application Started");
10        logger2.log(message: "Loading Modules");
11    }
12 }
```

Kiruthika@KIRUTHIKA-S MINGW64 ~/SingletonPatternExample
\$ javac Logger.java SingletonTest.java
• java SingletonTest
Logger instance created.
[LOG] Application Started
[LOG] Loading Modules
[LOG] Application Running

Logger1 HashCode : 1933863327
Logger2 HashCode : 1933863327
Logger3 HashCode : 1933863327

Only one Logger instance exists.
Kiruthika@KIRUTHIKA-S MINGW64 ~/SingletonPatternExample
o \$

Exercise 2 : Implementing the Factory Method Pattern

```
1 interface Document {
2     void open();
3 }
```

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• \$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• \$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
o \$

The screenshot shows an IDE with the Explorer panel on the left displaying a project structure for a Factory Method Pattern example. The main editor displays the `DocumentFactory.java` file, which defines an abstract class `DocumentFactory` with an abstract method `createDocument()`. The terminal at the bottom shows the execution of the program, which successfully creates and opens Word, PDF, and Excel documents.

```
EXPLORER
  FACTORY...
    Document.class
    Document.java
    DocumentFactory.class
    DocumentFactory.java
    ExcelDocument.class
    ExcelDocument.java
    ExcelFactory.class
    ExcelFactory.java
    FactoryTest.class
    FactoryTest.java
    PdfDocument.class
    PdfDocument.java
    PdfFactory.class
    PdfFactory.java
    WordDocument.class
    WordDocument.java
    WordFactory.class
    WordFactory.java

DocumentFactory.java
1  abstract class DocumentFactory {
2
3      abstract Document createDocument();
4
5  }
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$
```

The screenshot shows the same IDE with the `ExcelDocument.java` file open in the main editor. This class implements the `Document` interface and provides the `open()` method, which prints "Excel Document Opened" to the console. The terminal output shows the successful execution of the program, including the opening of the Excel document.

```
EXPLORER
  FACTORY...
    Document.class
    Document.java
    DocumentFactory.class
    DocumentFactory.java
    ExcelDocument.class
    ExcelDocument.java
    ExcelFactory.class
    ExcelFactory.java
    FactoryTest.class
    FactoryTest.java
    PdfDocument.class
    PdfDocument.java
    PdfFactory.class
    PdfFactory.java
    WordDocument.class
    WordDocument.java
    WordFactory.class
    WordFactory.java

ExcelDocument.java
1  class ExcelDocument implements Document {
2
3      public void open() {
4          System.out.println(x: "Excel Document Opened");
5      }
6
7  }
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
$
```

The screenshot shows an IDE with the Explorer view on the left displaying a project structure for a Factory Method Pattern example. The main editor displays the `ExcelFactory.java` file, which contains the following code:

```
1 class ExcelFactory extends DocumentFactory {  
2  
3     Document createDocument() {  
4         return new ExcelDocument();  
5     }  
6  
7 }
```

The bottom panel shows the TERMINAL view with the following output:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
o $
```

The screenshot shows the same IDE with the `FactoryTest.java` file open in the main editor. The code is as follows:

```
1 public class FactoryTest {  
2     public static void main(String[] args) {  
3  
4         Document d1 = f1.createDocument();  
5         d1.open();  
6  
7         DocumentFactory f2 = new PdfFactory();  
8         Document d2 = f2.createDocument();  
9         d2.open();  
10  
11         DocumentFactory f3 = new ExcelFactory();  
12         Document d3 = f3.createDocument();  
13         d3.open();  
14     }  
15  
16  
17  
18  
19 }
```

The bottom panel shows the TERMINAL view with the following output:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
o $
```

The screenshot shows an IDE with the Explorer on the left listing files like Document.class, DocumentFactory.class, ExcelDocument.class, and PdfDocument.class. The main editor displays PdfDocument.java with the following code:

```
1 class PdfDocument implements Document {  
2  
3     public void open() {  
4         System.out.println("PDF Document Opened");  
5     }  
6  
7 }
```

The bottom panel shows the Terminal output:

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
• $ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
o $
```

The screenshot shows the same IDE with PdfFactory.java selected in the Explorer and editor. The code is as follows:

```
1 class PdfFactory extends DocumentFactory {  
2  
3     Document createDocument() {  
4         return new PdfDocument();  
5     }  
6  
7 }
```

The Terminal output is identical to the previous screenshot, showing the successful compilation and execution of the FactoryTest program.

The screenshot shows an IDE with the Explorer panel on the left listing various Java files. The main editor displays the `WordDocument.java` file, which implements the `Document` interface. The code defines a `public void open()` method that prints "Word Document Opened". The terminal at the bottom shows the execution of `javac *.java` and `java FactoryTest`, resulting in the output "Word Document Opened", "PDF Document Opened", and "Excel Document Opened".

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println(x: "Word Document Opened");  
    }  
}
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$
```

The screenshot shows the same IDE with the Explorer panel listing Java files. The main editor displays the `WordFactory.java` file, which extends the `DocumentFactory` class. The code defines a `Document createDocument()` method that returns a new `WordDocument()`. The terminal at the bottom shows the execution of `javac *.java` and `java FactoryTest`, resulting in the output "Word Document Opened", "PDF Document Opened", and "Excel Document Opened".

```
class WordFactory extends DocumentFactory {  
    Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ javac *.java  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$ java FactoryTest  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
  
Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample  
$
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
• $ java FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

Kiruthika@KIRUTHIKA-S MINGW64 ~/FactoryMethodPatternExample
○ $
```

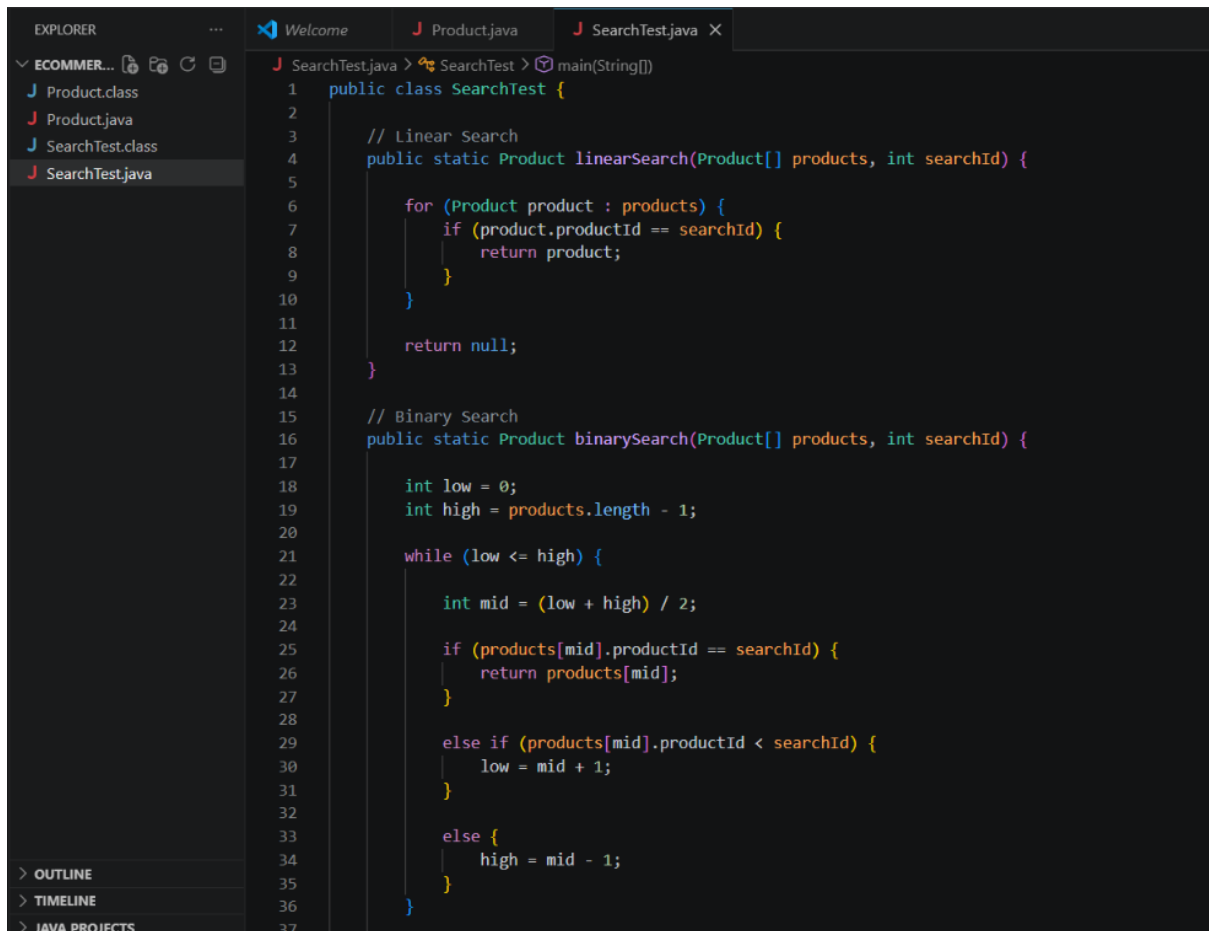
Data Structures and Algorithms

Exercise 2: E-commerce Platform Search Function

```
EXPLORER  ...  Welcome  J Product.java X  J SearchTest.java

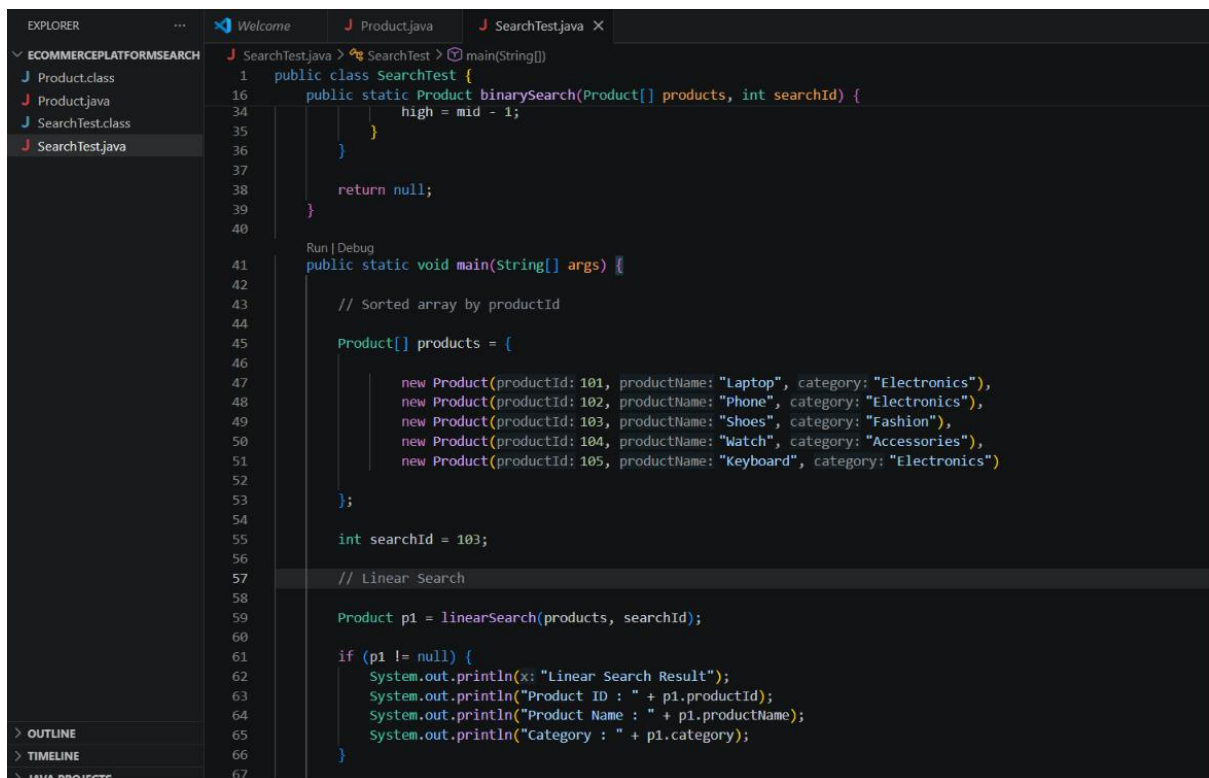
▼ ECOMMERCEPLATFORMSEARCH
J Product.class
J Product.java
J SearchTest.class
J SearchTest.java

J Product.java > Product
1  class Product {
2
3      int productId;
4      String productName;
5      String category;
6
7      Product(int productId, String productName, String category) {
8          this.productId = productId;
9          this.productName = productName;
10         this.category = category;
11     }
12 }
```

The screenshot shows an IDE with the Explorer on the left and the SearchTest.java file open in the editor. The code defines a SearchTest class with two static methods: linearSearch and binarySearch. The linearSearch method iterates through an array of Product objects to find a match by product ID. The binarySearch method uses a while loop to find the product ID in a sorted array.

```
1 public class SearchTest {
2
3     // Linear Search
4     public static Product linearSearch(Product[] products, int searchId) {
5
6         for (Product product : products) {
7             if (product.productId == searchId) {
8                 return product;
9             }
10        }
11
12        return null;
13    }
14
15    // Binary Search
16    public static Product binarySearch(Product[] products, int searchId) {
17
18        int low = 0;
19        int high = products.length - 1;
20
21        while (low <= high) {
22
23            int mid = (low + high) / 2;
24
25            if (products[mid].productId == searchId) {
26                return products[mid];
27            }
28
29            else if (products[mid].productId < searchId) {
30                low = mid + 1;
31            }
32
33            else {
34                high = mid - 1;
35            }
36        }
37    }
```



The screenshot shows the same IDE with the SearchTest.java file open. The code now includes a main method that creates a sorted array of Product objects and tests the linearSearch method. The array contains five products: Laptop, Phone, Shoes, Watch, and Keyboard. The searchId is set to 103, which corresponds to the 'Shoes' product.

```
1 public class SearchTest {
2
3     // Linear Search
4     public static Product linearSearch(Product[] products, int searchId) {
5
6         for (Product product : products) {
7             if (product.productId == searchId) {
8                 return product;
9             }
10        }
11
12        return null;
13    }
14
15    // Binary Search
16    public static Product binarySearch(Product[] products, int searchId) {
17
18        int low = 0;
19        int high = products.length - 1;
20
21        while (low <= high) {
22
23            int mid = (low + high) / 2;
24
25            if (products[mid].productId == searchId) {
26                return products[mid];
27            }
28
29            else if (products[mid].productId < searchId) {
30                low = mid + 1;
31            }
32
33            else {
34                high = mid - 1;
35            }
36        }
37    }
38
39    Run | Debug
40    public static void main(String[] args) {
41
42        // Sorted array by productId
43
44        Product[] products = {
45
46            new Product(productId: 101, productName: "Laptop", category: "Electronics"),
47            new Product(productId: 102, productName: "Phone", category: "Electronics"),
48            new Product(productId: 103, productName: "Shoes", category: "Fashion"),
49            new Product(productId: 104, productName: "Watch", category: "Accessories"),
50            new Product(productId: 105, productName: "Keyboard", category: "Electronics")
51        };
52
53        int searchId = 103;
54
55        // Linear Search
56
57        Product p1 = linearSearch(products, searchId);
58
59        if (p1 != null) {
60            System.out.println("Linear Search Result");
61            System.out.println("Product ID : " + p1.productId);
62            System.out.println("Product Name : " + p1.productName);
63            System.out.println("Category : " + p1.category);
64        }
65    }
66
67 }
```



```
1 public class SearchTest {
41     public static void main(String[] args) {
54
55         int searchId = 103;
56
57         // Linear Search
58
59         Product p1 = linearSearch(products, searchId);
60
61         if (p1 != null) {
62             System.out.println("Linear Search Result");
63             System.out.println("Product ID : " + p1.productId);
64             System.out.println("Product Name : " + p1.productName);
65             System.out.println("Category : " + p1.category);
66         }
67
68         System.out.println();
69
70         // Binary Search
71
72         Product p2 = binarySearch(products, searchId);
73
74         if (p2 != null) {
75             System.out.println("Binary Search Result");
76             System.out.println("Product ID : " + p2.productId);
77             System.out.println("Product Name : " + p2.productName);
78             System.out.println("Category : " + p2.category);
79         }
80     }
81 }
```

```
Kiruthika@KIRUTHIKA-S MINGW64 ~/EcommercePlatformSearch
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/EcommercePlatformSearch
$ java SearchTest
Linear Search Result
Product ID : 103
Product Name : Shoes
Category : Fashion

Binary Search Result
Product ID : 103
Product Name : Shoes
Category : Fashion

Kiruthika@KIRUTHIKA-S MINGW64 ~/EcommercePlatformSearch
$
```

Exercise 7: Financial Forecasting

```
EXPLORER
  FINANCIALFORECASTING
    FinancialForecast.class
    FinancialForecast.java

Welcome
FinancialForecast.java x
FinancialForecast > futureValue(double, double, int)
1 public class FinancialForecast {
2
3 // Recursive method to calculate future value
4 static double futureValue(double amount, double rate, int years) {
5
6     if (years == 0) {
7         return amount;
8     }
9
10    return futureValue(amount * (1 + rate), rate, years - 1);
11 }
12
Run | Debug
13 public static void main(String[] args) {
14
15     double amount = 10000; // Initial investment
16     double rate = 0.10;    // 10% annual growth
17     int years = 3;
18
19     double result = futureValue(amount, rate, years);
20
21     System.out.println("Future Value = " + result);
22 }
23 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Kiruthika@KIRUTHIKA-S MINGW64 ~/FinancialForecasting
$ javac *.java

Kiruthika@KIRUTHIKA-S MINGW64 ~/FinancialForecasting
$ java FinancialForecast
Future Value = 13310.000000000004

Kiruthika@KIRUTHIKA-S MINGW64 ~/FinancialForecasting
$

> OUTLINE
> TIMELINE
> JAVA PROJECTS
0 0 0 Java: Ready
```

Exercise 1: Control Structures

Scenario 1: Apply 1% Discount to Loan Interest Rates for Customers Above 60

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  🔄  ⌵  Aa  🗑️

1  CREATE TABLE Customers (
2      CustomerID NUMBER PRIMARY KEY,
3      Name VARCHAR2(30),
4      Age NUMBER,
5      Balance NUMBER,
6      IsVIP VARCHAR2(5)
7  );
8
9  CREATE TABLE Loans (
10     LoanID NUMBER PRIMARY KEY,
11     CustomerID NUMBER,
12     InterestRate NUMBER,
13     DueDate DATE,
14     CONSTRAINT fk_customer
15     FOREIGN KEY (CustomerID)
16     REFERENCES Customers (CustomerID)
17 );
18
19 INSERT INTO Customers VALUES (1, 'Ravi', 65, 15000, 'NO');
20 INSERT INTO Customers VALUES (2, 'Priya', 30, 8000, 'NO');
21 INSERT INTO Customers VALUES (3, 'Kumar', 70, 20000, 'NO');
22
23 INSERT INTO Loans VALUES (101, 1, 8, SYSDATE+15);
24 INSERT INTO Loans VALUES (102, 2, 9, SYSDATE+40);
25 INSERT INTO Loans VALUES (103, 3, 7, SYSDATE+20);
26
27 COMMIT;
28
29 BEGIN
30     FOR c IN (
31         SELECT c.CustomerID, l.LoanID
```

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  🔄  ⌵  Aa  🗑️

18
19 INSERT INTO Customers VALUES (1, 'Ravi', 65, 15000, 'NO');
20 INSERT INTO Customers VALUES (2, 'Priya', 30, 8000, 'NO');
21 INSERT INTO Customers VALUES (3, 'Kumar', 70, 20000, 'NO');
22
23 INSERT INTO Loans VALUES (101, 1, 8, SYSDATE+15);
24 INSERT INTO Loans VALUES (102, 2, 9, SYSDATE+40);
25 INSERT INTO Loans VALUES (103, 3, 7, SYSDATE+20);
26
27 COMMIT;
28
29 BEGIN
30     FOR c IN (
31         SELECT c.CustomerID, l.LoanID
32         FROM Customers c
33         JOIN Loans l
34         ON c.CustomerID = l.CustomerID
35         WHERE c.Age > 60
36     )
37     LOOP
38         UPDATE Loans
39         SET InterestRate = InterestRate - 1
40         WHERE LoanID = c.LoanID;
41     END LOOP;
42
43     COMMIT;
44 END;
45 /
46
47 SELECT * FROM Loans;
```

[SQL Worksheet]*       Aa 

```

22
23 INSERT INTO Loans VALUES (101,1,8,SYSDATE+15);
24 INSERT INTO Loans VALUES (102,2,9,SYSDATE+40);
25 INSERT INTO Loans VALUES (103,3,7,SYSDATE+20);
26
27 COMMIT;
28
29 BEGIN

```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.006 seconds

	LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	101	1	7	7/16/2026, 4:25:41
2	102	2	9	8/10/2026, 4:25:41
3	103	3	6	7/21/2026, 4:25:41

Scenario 2: Promote Customers to VIP

[SQL Worksheet]*      Aa 

```

1 CREATE TABLE Customers (
2     CustomerID NUMBER PRIMARY KEY,
3     Name VARCHAR2(30),
4     Age NUMBER,
5     Balance NUMBER,
6     IsVIP VARCHAR2(5)
7 );
8
9 INSERT INTO Customers VALUES (1,'Ravi',65,15000,'NO');
10 INSERT INTO Customers VALUES (2,'Priya',30,8000,'NO');
11 INSERT INTO Customers VALUES (3,'Kumar',70,20000,'NO');
12
13 COMMIT;
14
15 BEGIN
16     FOR c IN (
17         SELECT CustomerID
18         FROM Customers
19         WHERE Balance > 10000
20     )
21     LOOP
22         UPDATE Customers
23         SET IsVIP = 'YES'
24         WHERE CustomerID = c.CustomerID;
25     END LOOP;
26
27     COMMIT;
28 END;
29 /
30
31 SELECT * FROM Customers;

```

[SQL Worksheet]*

```

1 CREATE TABLE Customers (
2     CustomerID NUMBER PRIMARY KEY,
3     Name VARCHAR2(30),
4     Age NUMBER,
5     Balance NUMBER,
6     IsVIP VARCHAR2(5)
7 );
8

```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.01 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Ravi	65	15000	YES
2	2	Priya	30	8000	NO
3	3	Kumar	70	20000	YES

Scenario 3: Send Loan Due Reminders

[SQL Worksheet]*

```

1 SET SERVEROUTPUT ON;
2
3 CREATE TABLE Customers (
4     CustomerID NUMBER PRIMARY KEY,
5     Name VARCHAR2(30),
6     Age NUMBER,
7     Balance NUMBER,
8     IsVIP VARCHAR2(5)
9 );
10
11 CREATE TABLE Loans (
12     LoanID NUMBER PRIMARY KEY,
13     CustomerID NUMBER,
14     InterestRate NUMBER,
15     DueDate DATE,
16     CONSTRAINT fk_customer
17     FOREIGN KEY (CustomerID)
18     REFERENCES Customers (CustomerID)
19 );
20
21 INSERT INTO Customers VALUES (1, 'Ravi', 65, 15000, 'NO');
22 INSERT INTO Customers VALUES (2, 'Priya', 30, 8000, 'NO');
23 INSERT INTO Customers VALUES (3, 'Kumar', 70, 20000, 'NO');
24
25 INSERT INTO Loans VALUES (101, 1, 8, SYSDATE+15);
26 INSERT INTO Loans VALUES (102, 2, 9, SYSDATE+40);
27 INSERT INTO Loans VALUES (103, 3, 7, SYSDATE+20);
28
29 COMMIT;
30
31 BEGIN

```

```
[ SQL Worksheet]*
CREATE TABLE Loans (
    DueDate DATE,
    CONSTRAINT fk_customer
    FOREIGN KEY (CustomerID)
    REFERENCES Customers (CustomerID)
);

INSERT INTO Customers VALUES (1, 'Ravi', 65, 15000, 'NO');
INSERT INTO Customers VALUES (2, 'Priya', 30, 8000, 'NO');
INSERT INTO Customers VALUES (3, 'Kumar', 70, 20000, 'NO');

INSERT INTO Loans VALUES (101, 1, 8, SYSDATE+15);
INSERT INTO Loans VALUES (102, 2, 9, SYSDATE+40);
INSERT INTO Loans VALUES (103, 3, 7, SYSDATE+20);

COMMIT;

BEGIN
    FOR rec IN (
        SELECT c.Name, l.LoanID, l.DueDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID = l.CustomerID
        WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30
    )
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Dear ' || rec.Name ||
            ', your Loan ID ' || rec.LoanID ||
            ' is due on ' || TO_CHAR(rec.DueDate, 'DD-MON-YYYY')
        );
    END LOOP;
END;
```

[SQL Worksheet]*

CREATE TABLE Loans (
 DueDate DATE,
 CONSTRAINT fk_customer
 FOREIGN KEY (CustomerID)
 REFERENCES Customers (CustomerID)
);

INSERT INTO Customers VALUES (1, 'Ravi', 65, 15000, 'NO');

Query resultScript outputDBMS outputExplain PlanSQL history



Commit complete.
Elapsed: 00:00:00.001

SQL> BEGIN
FOR rec IN (
SELECT c.Name, l.LoanID, l.DueDate
FROM Customers c...

Show more...

Reminder: Dear Ravi, your Loan ID 101 is due on 16-JUL-2026
Reminder: Dear Kumar, your Loan ID 103 is due on 21-JUL-2026

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.021

Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest

```
[ SQL Worksheet ]*
1 CREATE TABLE Accounts (
2     AccountNo NUMBER PRIMARY KEY,
3     AccountType VARCHAR2(20),
4     Balance NUMBER
5 );
6
7 INSERT INTO Accounts VALUES (101, 'SAVINGS', 10000);
8 INSERT INTO Accounts VALUES (102, 'SAVINGS', 5000);
9 INSERT INTO Accounts VALUES (103, 'CURRENT', 20000);
10
11 COMMIT;
12
13 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
14 AS
15 BEGIN
16     UPDATE Accounts
17     SET Balance = Balance + (Balance * 0.01)
18     WHERE AccountType = 'SAVINGS';
19
20     COMMIT;
21 END;
22 /
23
24 BEGIN
25     ProcessMonthlyInterest;
26 END;
27 /
28
29 SELECT * FROM Accounts;
```

[SQL Worksheet]*

```
1 CREATE TABLE Accounts (
2     AccountNo NUMBER PRIMARY KEY,
3     AccountType VARCHAR2(20),
4     Balance NUMBER
5 );
6
7 INSERT INTO Accounts VALUES (101, 'SAVINGS', 10000);
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.007 seconds

	ACCOUNTNO	ACCOUNTTYPE	BALANCE
1	101	SAVINGS	10100
2	102	SAVINGS	5050
3	103	CURRENT	20000

Scenario 2: UpdateEmployeeBonus

Scenario 2: Update Employee Bonus

```
[ SQL Worksheet ]*      Aa 
```

```
1 CREATE TABLE Employees (  
2     EmpID NUMBER PRIMARY KEY,  
3     Name VARCHAR2(30),  
4     Department VARCHAR2(30),  
5     Salary NUMBER  
6 );  
7  
8 INSERT INTO Employees VALUES (1, 'Ravi', 'IT', 50000);  
9 INSERT INTO Employees VALUES (2, 'Priya', 'HR', 40000);  
10 INSERT INTO Employees VALUES (3, 'Kumar', 'IT', 60000);  
11  
12 COMMIT;  
13  
14 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (  
15     p_department IN VARCHAR2,  
16     p_bonus IN NUMBER  
17 )  
18 AS  
19 BEGIN  
20     UPDATE Employees  
21     SET Salary = Salary + (Salary * p_bonus / 100)  
22     WHERE Department = p_department;  
23  
24     COMMIT;  
25 END;  
26 /  
27  
28 BEGIN  
29     UpdateEmployeeBonus ('IT', 10);  
30 END;  
31 /  
32  
33 SELECT * FROM Employees;
```

[SQL Worksheet]* Aa

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.009 seconds

	EMPID	NAME	DEPARTMENT	SALARY
1	1	Ravi	IT	55000
2	2	Priya	HR	40000
3	3	Kumar	IT	66000

Scenario 3: Transfer Funds

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  📄  ⌵  Aa  🗑️

1  CREATE TABLE Accounts (
2      AccountNo NUMBER PRIMARY KEY,
3      Balance NUMBER
4  );
5
6  INSERT INTO Accounts VALUES (101,10000);
7  INSERT INTO Accounts VALUES (102,5000);
8
9  COMMIT;
10
11 CREATE OR REPLACE PROCEDURE TransferFunds(
12     p_from IN NUMBER,
13     p_to IN NUMBER,
14     p_amount IN NUMBER
15 )
16 AS
17     v_balance NUMBER;
18 BEGIN
19     SELECT Balance
20     INTO v_balance
21     FROM Accounts
22     WHERE AccountNo = p_from;
23
24     IF v_balance >= p_amount THEN
25
26         UPDATE Accounts
27         SET Balance = Balance - p_amount
28         WHERE AccountNo = p_from;
29
30         UPDATE Accounts
31         SET Balance = Balance + p_amount
32         WHERE AccountNo = p_to;
33
34     COMMIT;
```

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  📄  ⌵  Aa  🗑️

6  INSERT INTO Accounts VALUES (101,10000);
7  INSERT INTO Accounts VALUES (102,5000);
8
9  COMMIT;
10
11 CREATE OR REPLACE PROCEDURE TransferFunds(
12     p_from IN NUMBER,
13     p_to IN NUMBER,
14     p_amount IN NUMBER
15 )
16 AS
17     v_balance NUMBER;
18 BEGIN
19     SELECT Balance
20     INTO v_balance
21     FROM Accounts
22     WHERE AccountNo = p_from;
23
24     IF v_balance >= p_amount THEN
25
26         UPDATE Accounts
27         SET Balance = Balance - p_amount
28         WHERE AccountNo = p_from;
29
30         UPDATE Accounts
31         SET Balance = Balance + p_amount
32         WHERE AccountNo = p_to;
33
34         COMMIT;
35
36         DBMS_OUTPUT.PUT_LINE('Transfer Successful');
37
38     ELSE
39
40     END IF;
```

```
[ SQL Worksheet ]*  ▶  ⌵  🔍  ↻  ⌵  Aa  🗑️

18  BEGIN
23
24      IF v_balance >= p_amount THEN
25
26          UPDATE Accounts
27          SET Balance = Balance - p_amount
28          WHERE AccountNo = p_from;
29
30          UPDATE Accounts
31          SET Balance = Balance + p_amount
32          WHERE AccountNo = p_to;
33
34          COMMIT;
35
36          DBMS_OUTPUT.PUT_LINE('Transfer Successful');
37
38      ELSE
39
40          DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
41
42      END IF;
43  END;
44  /
45
46  BEGIN
47      TransferFunds(101,102,2000);
48  END;
49  /
50
51  SELECT * FROM Accounts;
```

[SQL Worksheet]* ▶ ⌵ 🔍 ↻ ⌵ Aa 🗑️

18 BEGIN
23
24 IF v_balance >= p_amount THEN
25
26 UPDATE Accounts
27 SET Balance = Balance - p_amount
28 WHERE AccountNo = p_from;
29

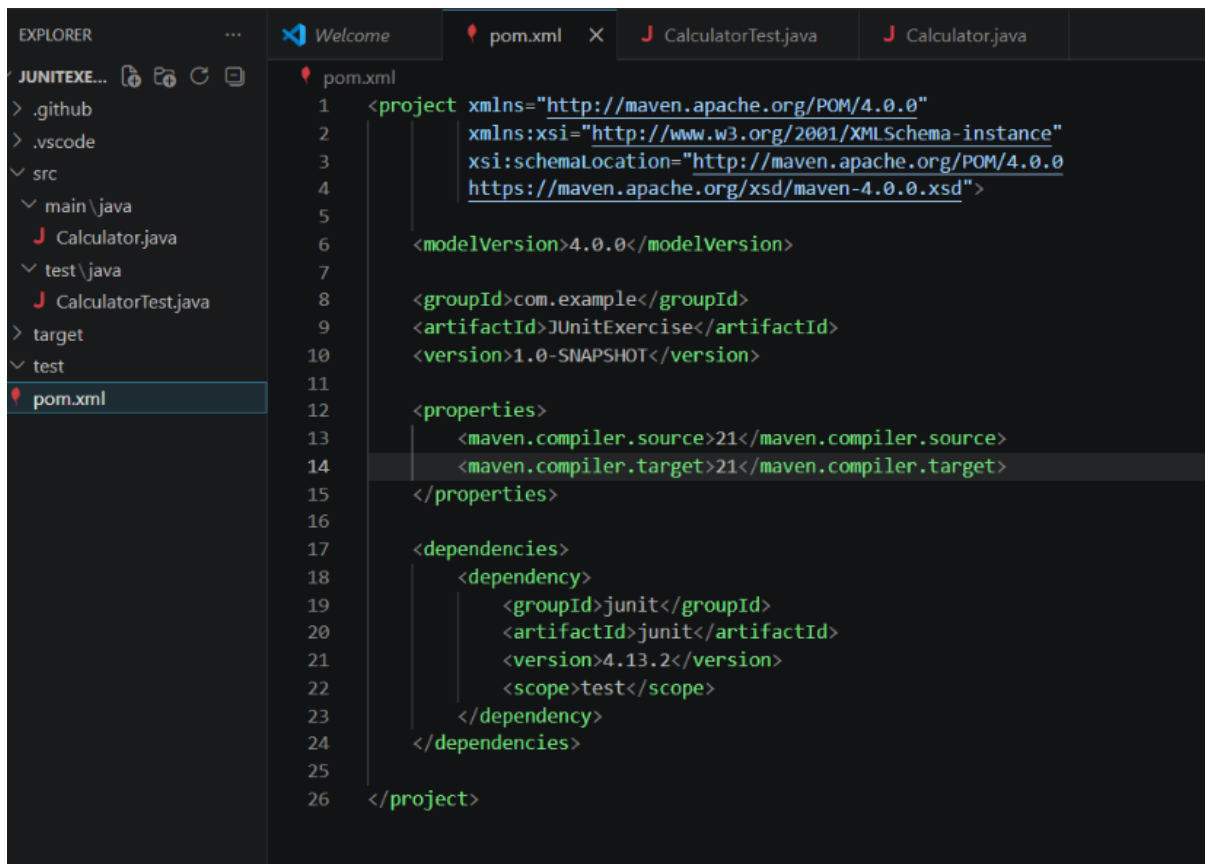
Query result Script output DBMS output Explain Plan SQL history

🗑️ ⓘ Download ▾ Execution time: 0.001 seconds

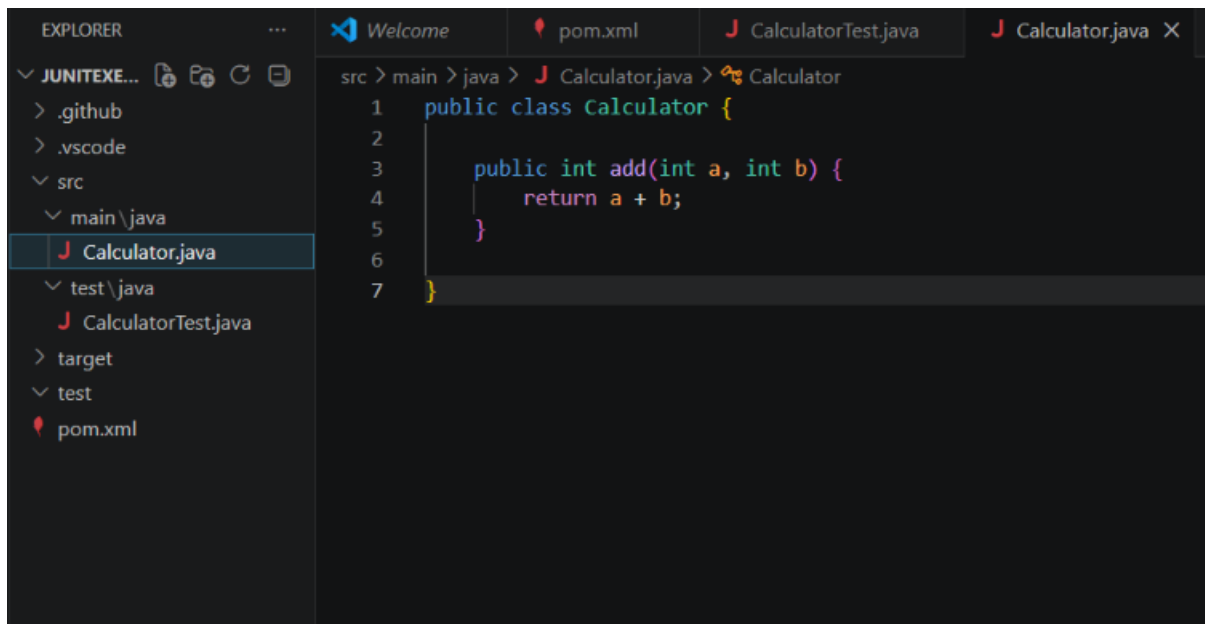
	ACCOUNTNO	ACCOUNTTYPE	BALANCE
1	101	SAVINGS	8100
2	102	SAVINGS	7050
3	103	CURRENT	20000

TDD using JUnit5 and Mockito

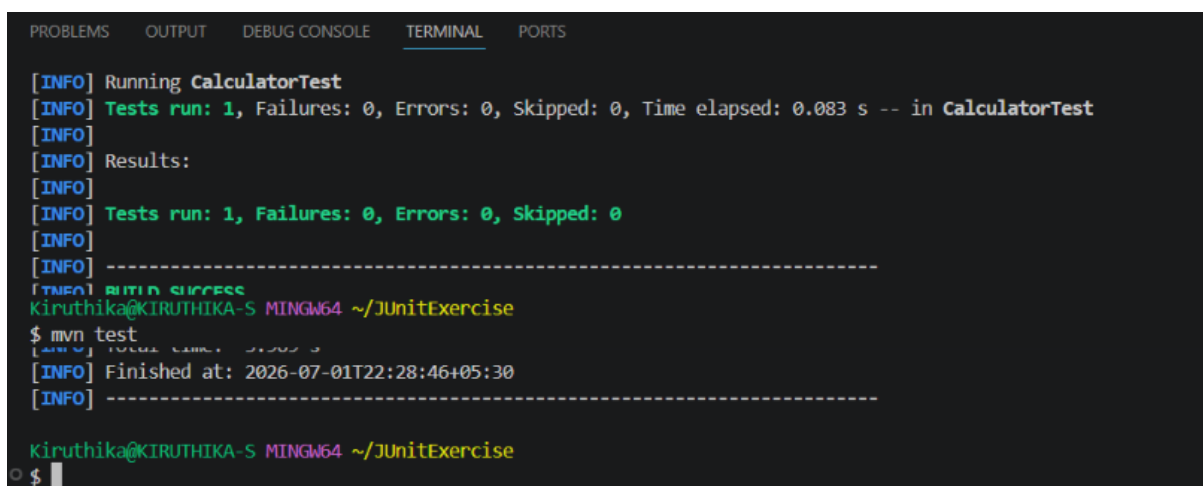
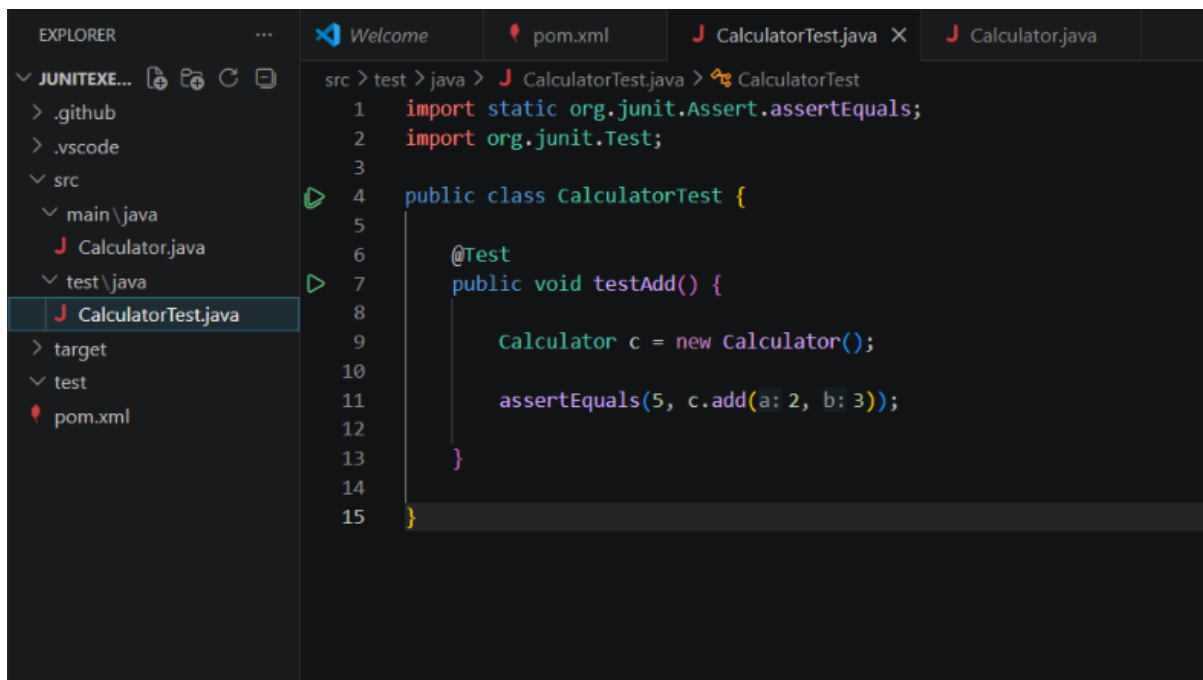
Exercise 1: Setting Up Junit



```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4       https://maven.apache.org/xsd/maven-4.0.0.xsd">
5
6     <modelVersion>4.0.0</modelVersion>
7
8     <groupId>com.example</groupId>
9     <artifactId>JUnitExercise</artifactId>
10    <version>1.0-SNAPSHOT</version>
11
12    <properties>
13      <maven.compiler.source>21</maven.compiler.source>
14      <maven.compiler.target>21</maven.compiler.target>
15    </properties>
16
17    <dependencies>
18      <dependency>
19        <groupId>junit</groupId>
20        <artifactId>junit</artifactId>
21        <version>4.13.2</version>
22        <scope>test</scope>
23      </dependency>
24    </dependencies>
25
26  </project>
```



```
1 public class Calculator {
2
3     public int add(int a, int b) {
4         return a + b;
5     }
6
7 }
```



Exercise 3: Assertions in Junit

AssertionsTest.java

```
import static org.junit.Assert.*;
```

```
import org.junit.Test;
```

```
public class AssertionsTest {
```

```
    @Test
```

```
    public void testAssertions() {
```

```
    assertEquals(5, 2 + 3);
    assertTrue(5 > 3);
    assertFalse(5 < 3);
    assertNull(null);
    assertNotNull(new Object());
}
}
```

OUTPUT:

T E S T S

Running AssertionsTest

Tests run: 1

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in Junit

Calculator.java

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

```
}
```

CalculatorTest.java

```
import static org.junit.Assert.assertEquals;
```

```
import org.junit.After;
```

```
import org.junit.Before;
```

```
import org.junit.Test;
```

```
public class CalculatorTest {
```

```
    private Calculator calculator;
```

```
    @Before
```

```
    public void setUp() {
```

```
        calculator = new Calculator();
```

```
        System.out.println("Setup completed");
```

```
    }
```

```
    @Test
```

```
    public void testAdd() {
```

```
        // Arrange
```

```
        int a = 2;
```

```
        int b = 3;
```

```
        // Act
```



```

    int result = calculator.add(a, b);

    // Assert
    assertEquals(5, result);
}

@After
public void tearDown() {

    calculator = null;

    System.out.println("Teardown completed");
}
}

```

pom.xml

```

<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>

```

OUTPUT:

Setup completed

Teardown completed

Tests run: 1

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

Mockito exercises

Exercise 1: Mocking and Stubbing

pom.xml

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.12.0</version>
  <scope>test</scope>
</dependency>
```

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
```

ExternalApi.java

```
public interface ExternalApi {
```

```
    String getData();
```

```
}
```

MyService.java

```
public class MyService {  
  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
  
    public String fetchData() {  
        return api.getData();  
    }  
  
}
```

MyServiceTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
import static org.mockito.Mockito.when;  
  
import org.junit.jupiter.api.Test;  
import org.mockito.Mockito;  
  
public class MyServiceTest {  
  
    @Test  
    public void testExternalApi() {
```

```
ExternalApi mockApi = Mockito.mock(ExternalApi.class);

when(mockApi.getData()).thenReturn("Mock Data");

MyService service = new MyService(mockApi);

String result = service.fetchData();

assertEquals("Mock Data", result);
}
}
```

OUTPUT:

T E S T S

Running MyServiceTest

Tests run: 1

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

Exercise 2: Verifying Interactions

pom.xml

<dependency>

 <groupId>org.junit.jupiter</groupId>

```
<artifactId>junit-jupiter</artifactId>
<version>5.10.2</version>
<scope>test</scope>
</dependency>

<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.12.0</version>
  <scope>test</scope>
</dependency>
```

ExternalApi.java

```
public interface ExternalApi {

    String getData();

}
```

MyService.java

```
public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}
```

```

    }

}

MyServiceTest.java

import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testVerifyInteraction() {

        // Create mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Create service object
        MyService service = new MyService(mockApi);

        // Call the method
        service.fetchData();

        // Verify interaction
        verify(mockApi).getData();
    }
}

```

OUTPUT:

T E S T S

Running MyServiceTest

Tests run: 1

Failures: 0

Errors: 0

Skipped: 0

BUILD SUCCESS

SL4J Logging exercises

Exercise 1: Logging Error Messages and Warning Levels

pom.xml

```
<dependency>
```

```
  <groupId>org.slf4j</groupId>
```

```
  <artifactId>slf4j-api</artifactId>
```

```
  <version>1.7.30</version>
```

```
</dependency>
```

```
<dependency>
```

```
  <groupId>ch.qos.logback</groupId>
```

```
  <artifactId>logback-classic</artifactId>
```

```
  <version>1.2.3</version>
```

```
</dependency>
```

LoggingExample.java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {

        logger.error("This is an error message");

        logger.warn("This is a warning message");

    }
}
```

OUTPUT:

ERROR LoggingExample - This is an error message

WARN LoggingExample - This is a warning message